

Personal Finance Tracker

**A PROJECT REPORT
for
Major Project-II (CA401P)
Session (2025-26)**

**Submitted by
Aanchal
202410116100002
Bobby Karnik
202410116100051
Brijesh Sharma
202410116100052
Devesh Alan
202410116100061**

**Submitted in partial fulfilment of the
Requirements for the Degree of**

MASTER OF COMPUTER APPLICATIONS

**Under the Supervision of
Ms. Sonam Jain
Assistant Professor**



**Submitted to
DEPARTMENT OF COMPUTER APPLICATIONS
KIET Group of Institutions, Ghaziabad
Uttar Pradesh-201206
(MAY- 2026)**

CERTIFICATE

Certified that **Aanchal 202410116100002, Devesh Alan 202410116100061, Bobby Karnik 202410116100051, Brijesh Sharma 202410116100052** have carried out the project work having “**Title of Report: Personal Finance Tracker**” (CA401P) for **Master of Computer Applications** from KIET Group of Institutions (an Autonomous college) under Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself/herself and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Date:

Aanchal (202410116100002)

Bobby Karnik (202410116100051)

Devesh Alan (202410116100061)

Brijesh Sharma (202410116100052)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Ms. Sonam Jain
Assistant Professor
Department of Computer Applications
KIET Group of Institutions, Ghaziabad

Dr. Sachin Malhotra
Dean (MCA)
Department of Computer Applications
KIET Group of Institutions, Ghaziabad

Problem Statement

Parents face considerable challenges in educating their children through storytelling, particularly when striving to meet the diverse developmental, cognitive, and accessibility needs of young learners. Traditional storytelling methods are often not inclusive, making it difficult to engage differently-abled children—such as those with visual or hearing impairments or neurodivergent learning styles. This lack of adaptive content limits equal participation and restricts the educational value of storytelling. Furthermore, parents struggle to find personalized, growth-oriented storytelling approaches that cater to varying learning speeds and developmental stages, leading to uneven educational experiences.

The integration of technology into storytelling has the potential to address these challenges, yet many existing digital tools are neither user-friendly nor accessibility-focused. Parents often lack the technical expertise to customize these platforms to suit their children's unique needs. Additionally, striking the right balance between entertainment and education remains a persistent challenge without access to expertly designed, interactive storytelling environments.

This presents an opportunity for a full stack development solution: an intuitive, customizable, and accessible digital storytelling platform. Such a platform can leverage backend intelligence and frontend adaptability to provide inclusive, interactive, and educational storytelling experiences tailored to every child's learning style and ability.

Keywords: Full Stack Development, Storytelling, Inclusivity, Accessibility, Personalized Learning, Technology Integration.

ACKNOWLEDGEMENTS

Success in life is never attained single-handedly. My deepest gratitude goes to my project supervisor, **Ms. Sonam Jain** for her guidance, help, and encouragement throughout my project work. Their enlightening ideas, comments, and suggestions.

Words are not enough to express my gratitude to **Dr. Sachin Malhotra, Professor and Dean, Department of Computer Applications**, for his insightful comments and administrative help on various occasions.

Fortunately, I have many understanding friends, who have helped me a lot in many critical conditions.

Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kinds of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Aanchal

Bobby Karnik

Devesh Alan

Brijesh Sharma

TABLE OF CONTENTS

Certificate	ii
Abstract	iii
Acknowledgements	iv
Table of Contents	v
1 Introduction	1-9
1.1 Overview	1
1.2 Problem Statement	2
1.3 Learning Capabilities	3-5
1.4 Growth Orientation Study	5-7
1.5 Scope of the Project	7-9
2 Feasibility Study	10-15
2.1 Technical Feasibility	10-12
2.2 Economic Feasibility	12-13
2.3 Operational Feasibility	13-14
2.4 Social Feasibility	14-15
3 Project Objective	16-17
4 Hardware and Software Requirements	18-19
5 Project Flow	20-24
6 Project Outcome	25-88
References	89-92

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In the modern digital era, managing personal finances has become increasingly complex due to the rise in digital transactions, online shopping platforms, subscription services, and diverse income sources. Individuals often struggle to maintain a clear understanding of their financial activities, leading to poor budgeting, unnecessary expenditures, and inadequate savings.

The **AI-Powered Personal Finance Tracker** is designed as an advanced financial management system that aims to simplify and automate the process of tracking income and expenses. The system leverages modern web technologies along with Artificial Intelligence (AI) and Machine Learning (ML) techniques to provide users with intelligent insights into their financial behavior.

The platform allows users to:

- Record daily income and expenses
- Categorize transactions into predefined or custom categories
- Upload bills and receipts for automated data extraction
- Visualize financial data using graphs and charts
- Predict future expenses based on historical data

One of the most significant features of the system is the integration of **Optical Character Recognition (OCR)** technology. This feature enables users to upload images of bills or receipts, from which the system automatically extracts relevant information such as:

- Total amount
- Date of transaction
- Items purchased

This eliminates the need for manual data entry and ensures accuracy and efficiency.

Additionally, the system incorporates **AI-based predictive models** that analyze historical financial data to forecast future expenses. These predictions help users plan their budgets more effectively and avoid financial instability.

The application is developed using the **MERN Stack**, which includes:

- MongoDB (Database)
- Express.js (Backend Framework)
- React.js (Frontend Framework)
- Node.js (Runtime Environment)

This architecture ensures:

- High scalability
- Efficient performance
- Smooth user interaction

Overall, the AI-Powered Personal Finance Tracker serves as a comprehensive solution for individuals seeking to gain control over their finances and develop better financial habits.

1.2 Problem Statement

In today's fast-paced and digitally driven environment, individuals frequently engage in numerous financial transactions on a daily basis. These transactions include online purchases, subscription payments, utility bills, and miscellaneous expenses. Despite the availability of various financial tools and applications, many users still face significant challenges in effectively managing their personal finances.

One of the primary issues is the reliance on **manual expense tracking**, which is both time-consuming and prone to human error. Users often forget to record small transactions, leading to incomplete financial records and inaccurate analysis. Furthermore, existing applications typically require users to manually categorize expenses, which adds to the complexity and discourages consistent usage.

Another major challenge is the **lack of predictive insights**. Most traditional finance tracking tools only provide historical data without offering guidance on future spending patterns. As a result, users are unable to anticipate upcoming financial needs or identify potential overspending trends in advance.

Additionally, users struggle with:

- Identifying unnecessary or excessive spending
- Managing recurring subscriptions
- Understanding category-wise expenditure distribution
- Setting realistic financial goals

There is also a **lack of automation and intelligence** in many existing systems. Users are required to perform repetitive tasks, which reduces efficiency and increases the chances of errors.

Therefore, there is a strong need for an intelligent system that:

- Automates financial data entry
- Provides real-time insights
- Predicts future expenses
- Offers actionable suggestions

The AI-Powered Personal Finance Tracker addresses these challenges by integrating AI, OCR, and data analytics to create a smart, efficient, and user-friendly financial management platform.

1.3 Learning Capabilities and Developmental Needs

The AI-Powered Personal Finance Tracker is not only a financial management tool but also a platform that enhances users' financial literacy and decision-making capabilities.

1.3.1 Financial Awareness

The system enables users to understand their spending habits through:

- Category-wise expense tracking
- Monthly and yearly summaries
- Visual representation of financial data

This helps users identify where their money is being spent and make informed decisions.

1.3.2 Self-Paced Learning

Users can analyze their financial data at their own pace without any time constraints. They can:

- Review past transactions
- Compare spending trends
- Evaluate financial goals

1.3.3 AI-Based Learning

The AI engine provides insights such as:

- Spending increase alerts
- Budget recommendations
- Savings suggestions

These insights help users learn from their financial behavior and improve over time.

1.3.4 Practical Financial Skills

The platform promotes:

- Budget planning
- Expense control
- Savings management
- Financial discipline

1.4 Growth-Oriented Study

The system is designed with a strong focus on long-term growth and scalability.

1.4.1 User Growth

The platform encourages users to:

- Develop consistent financial habits
- Improve saving patterns
- Reduce unnecessary expenditures

1.4.2 Technological Growth

The system architecture supports future enhancements such as:

- Integration with banking APIs
- Advanced AI models
- Real-time analytics

1.4.3 Feature Expansion

Future features may include:

- Investment tracking
- Loan management
- Credit score analysis
- Tax planning assistance

1.4.4 Institutional Growth

The system can be extended for:

- Financial education programs
- Enterprise expense management
- Multi-user family accounts

1.5 Scope of the Project (Expanded)

1.5.1 Current Scope

The project currently focuses on:

- Income and expense tracking
- Automated bill processing using OCR
- AI-based expense prediction
- Data visualization through charts
- Spending pattern analysis

1.5.2 Functional Scope

The system includes:

- User authentication
- Transaction management
- AI insight generation
- Reporting and analytics

1.5.3 Limitations

- No direct integration with bank accounts
- Predictions depend on available data
- Limited offline functionality

1.5.4 Future Scope

The system can be expanded to include:

- Mobile application development
- Real-time financial synchronization
- AI-based financial assistant
- Multi-language support
- Global currency support

CHAPTER 2

FEASIBILITY STUDY & PROJECT APPROACH

2.1 Introduction to Feasibility Study

A feasibility study is a crucial phase in system development that evaluates the practicality, viability, and potential success of a proposed project. It helps determine whether the system can be developed within the available resources, constraints, and technological capabilities.

For the **AI-Powered Personal Finance Tracker**, the feasibility study examines multiple aspects such as:

- Technical feasibility
- Economic feasibility
- Operational feasibility
- Social feasibility
- Legal feasibility (extended)
- Schedule feasibility

The purpose of this study is to ensure that the system is:

- Technically implementable
- Cost-effective
- User-friendly
- Beneficial to society

By analyzing these factors, developers can identify potential risks, limitations, and improvements before full-scale implementation.

2.2 Technical Feasibility

Technical feasibility determines whether the required technology, tools, and resources are available to develop and deploy the system successfully.

2.2.1 Technology Stack Feasibility

The project uses modern and widely adopted technologies:

- **Frontend:** React.js
- **Backend:** Node.js with Express.js
- **Database:** MongoDB
- **AI/ML Models:** Linear Regression, Time Series (ARIMA/LSTM)
- **OCR Technology:** Tesseract OCR / API-based OCR

These technologies are:

- Open-source
- Well-documented
- Highly scalable
- Supported by large developer communities

This ensures smooth development, debugging, and future upgrades.

2.2.2 System Architecture Feasibility

The system follows a **client-server architecture**, where:

- The frontend handles user interaction
- The backend processes logic and API requests
- The database stores financial data

Advantages:

- Separation of concerns
- Easy maintenance
- Scalability for future expansion

2.2.3 AI Model Feasibility

The AI models used for prediction are feasible because:

- They require structured historical data
- They can be trained using standard ML libraries
- They provide accurate short-term predictions

Example:

- Linear Regression → Predict monthly expenses
- Time Series Models → Forecast trends

2.2.4 OCR Feasibility

OCR integration is technically feasible because:

- Pre-built APIs and libraries are available
- Can process images efficiently
- Reduces manual work significantly

2.2.5 Hardware and Software Compatibility

The system runs on:

- Standard laptops/desktops
- Modern browsers
- Cross-platform OS (Windows, Linux, macOS)

No high-end infrastructure is required, making implementation easy.

2.2.6 Technical Challenges and Solutions

Challenge	Solution
Inaccurate OCR extraction	Use preprocessing and validation
Data inconsistency	Input validation and cleaning
Prediction accuracy	Use improved ML models
Performance issues	Optimize APIs and database queries

2.3 Economic Feasibility

Economic feasibility evaluates the cost-effectiveness of the project.

2.3.1 Development Cost

The project is highly economical because it uses:

- Open-source technologies
- Free development tools

- Minimal infrastructure

Cost components:

- Development time
- Hosting (optional)
- Maintenance

2.3.2 Operational Cost

Operational costs are minimal:

- Cloud hosting (optional)
- Database storage
- API usage (OCR/AI)

2.3.3 Cost-Benefit Analysis

Cost	Benefit
Low development cost	High user value
Minimal infrastructure	Scalable system
Free tools	No licensing cost

2.3.4 Long-Term Economic Benefits

- Saves user time
- Reduces financial mismanagement
- Improves savings

Thus, the project is economically viable.

2.4 Operational Feasibility

Operational feasibility determines whether the system can be used effectively by end users.

2.4.1 User-Friendly Interface

The system is designed with:

- Simple navigation
- Clean UI
- Easy interaction

Users can:

- Add transactions easily
- View reports instantly
- Upload bills without complexity

2.4.2 Ease of Use

The platform requires:

- No technical knowledge
- Minimal training
- Simple login and dashboard access

2.4.3 System Reliability

The system ensures:

- Accurate data processing
- Secure authentication
- Stable performance

2.4.4 User Acceptance

The system is likely to be accepted because:

- It solves real-life problems
- Saves time
- Provides intelligent insights

2.4.5 Operational Challenges

- Users may forget to update data

- OCR errors in poor image quality

Solutions:

- Notifications/reminders
- Image preprocessing

2.5 Social Feasibility

Social feasibility evaluates the impact of the system on society.

2.5.1 Positive Social Impact

The system promotes:

- Financial awareness
- Responsible spending
- Better savings habits

2.5.2 Educational Value

It helps users:

- Learn budgeting
- Understand financial planning
- Improve decision-making

2.5.3 Behavioral Impact

Encourages:

- Discipline
- Accountability
- Smart financial choices

2.5.4 Accessibility

The system is:

- Easy to access

- Usable by students and professionals
- Inclusive for all age groups

2.6 Legal Feasibility

Legal feasibility ensures compliance with laws and regulations.

2.6.1 Data Privacy

The system ensures:

- Secure storage of user data
- No unauthorized sharing
- Encryption of sensitive information

2.6.2 Compliance

The system follows:

- Data protection guidelines
- Ethical AI usage

2.6.3 Intellectual Property

- Open-source tools are used legally
- No copyrighted data misuse

2.7 Schedule Feasibility

Schedule feasibility evaluates whether the project can be completed within a given time frame.

2.7.1 Development Phases

1. Requirement Analysis
2. System Design
3. Development
4. Testing
5. Deployment

PROJECT FLOW – AI-POWERED PERSONAL FINANCE TRACKER

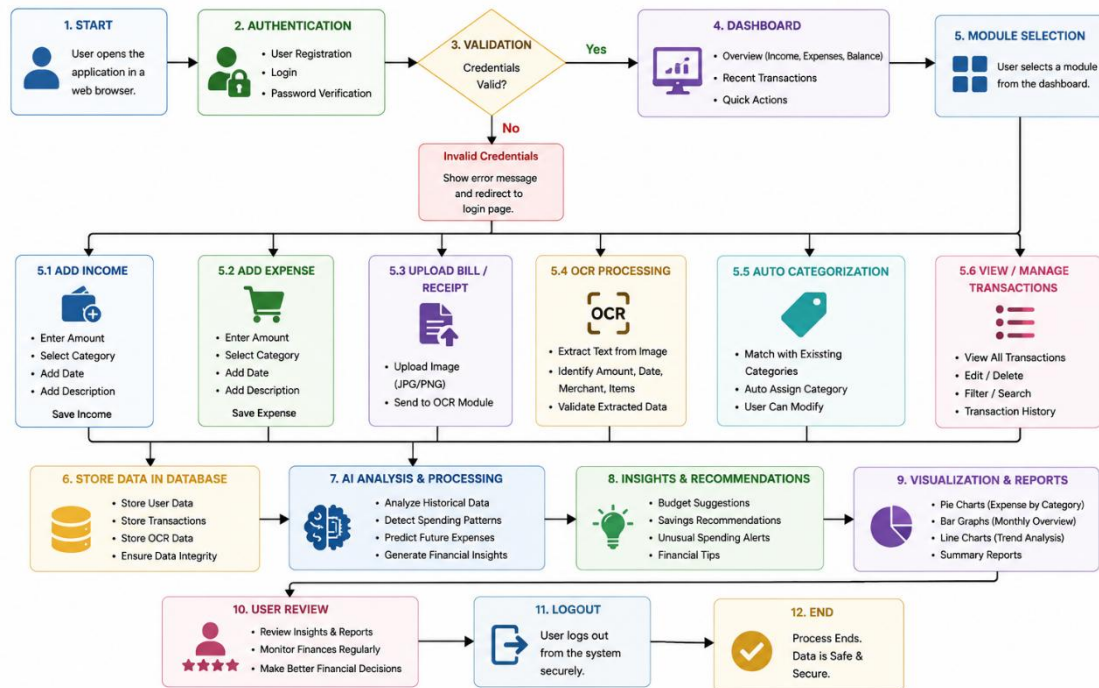


Figure 1: Project Flow

CHAPTER 3

Project Objective

3.1 PROJECT OBJECTIVE

Project objectives define the overall goals and intended outcomes of a software system. They provide a clear direction for development and help ensure that all modules and functionalities are aligned with the purpose of the project. The objectives of a project also help in identifying the problems to be solved and the expected benefits for users.

The **Personal Finance Tracker** – Senior–Junior Connection Platform is developed with the primary objective of creating a structured, reliable, and collaborative placement preparation environment for students. The project aims to bridge the communication gap between senior and junior students by enabling organized knowledge sharing, mentorship, and access to authentic placement resources.

In today's competitive academic and professional environment, students often face difficulties during placement preparation due to scattered resources, lack of guidance, and insufficient awareness regarding interview processes. The Personal Finance Tracker platform is designed to overcome these challenges by providing a centralized digital ecosystem where seniors can share their experiences and juniors can learn in a systematic and efficient manner.

The project focuses not only on technical implementation but also on improving student confidence, collaboration, and placement readiness. The objectives of the project are explained below in detail.

Main Objective of the Project

The primary objective of the Personal Finance Tracker platform is to establish a strong and structured connection between senior and junior students for placement preparation and career guidance. The system aims to simplify the preparation process by organizing placement-related information and making it easily accessible to all students within the academic institution.

The project seeks to create a platform where seniors who have already participated in placement drives can share their practical experiences, preparation methods, interview

patterns, and technical guidance. Juniors can use this information to understand company expectations and prepare effectively for upcoming recruitment opportunities.

The system is designed to improve placement readiness, reduce confusion, and create a supportive mentorship environment that promotes collaborative learning and peer guidance.

Objectives of the Personal Finance Tracker Platform

1. To Provide Structured Placement Guidance

One of the major objectives of the project is to organize placement-related resources in a structured and systematic manner. Many students face difficulty in identifying relevant preparation materials because online resources are scattered and unorganized.

The Personal Finance Tracker platform aims to solve this issue by categorizing resources such as:

- Company-wise interview questions
- Technical preparation materials
- Guidance videos
- Placement experiences
- Programming language questions
- Quiz and assessment modules

This structured organization helps students focus on targeted preparation instead of wasting time searching for scattered information.

2. To Bridge the Gap Between Seniors and Juniors

The platform is developed to reduce the communication gap between experienced senior students and juniors who are beginning their placement journey. Seniors possess valuable practical knowledge regarding recruitment processes, coding rounds, aptitude tests, and HR interviews.

The objective of the project is to create a dedicated platform where this knowledge can be shared efficiently and systematically. Juniors can directly learn from seniors who have already faced placement interviews, making preparation more realistic and practical.

The project encourages:

- Peer mentorship
- Experience sharing
- Collaborative learning
- Guidance-based preparation

This interaction creates a stronger academic support system within the institution.

3. To Improve Placement Preparation Efficiency

Another important objective is to improve the efficiency of placement preparation by providing students with authentic and relevant guidance. The platform minimizes dependency on unreliable online resources and helps students prepare according to actual company recruitment patterns.

The project aims to save students' time and effort by providing all essential placement-related resources at a single platform. Students can access preparation content according to their targeted companies, technical domains, or learning requirements.

This objective helps students:

- Prepare in a focused manner
- Understand interview trends
- Improve technical knowledge
- Enhance confidence during interviews

4. To Promote Knowledge Sharing and Peer Learning

The Personal Finance Tracker platform encourages students to actively participate in knowledge sharing and collaborative learning. Seniors contribute by uploading guidance videos, preparation strategies, and interview experiences, while juniors engage through discussions and query-resolution activities.

The objective is to create a continuous learning environment where students learn from one another rather than depending only on traditional classroom teaching. This promotes academic interaction and strengthens the culture of mentorship within the institution.

The platform supports:

- Discussion forums
- Query-resolution systems
- Shared preparation resources
- Collaborative problem-solving

This peer-learning model helps students improve both technical and communication skills.

5. To Provide Company-Specific Interview Preparation

Placement preparation varies according to company requirements, recruitment patterns, and job roles. Therefore, one of the key objectives of the project is to provide company-oriented preparation support.

The platform organizes interview questions and preparation experiences according to specific companies and technical domains. This helps students understand:

- Company interview patterns

- Frequently asked questions
- Technical expectations
- HR interview strategies
- Coding round preparation methods

Company-specific preparation increases the effectiveness of learning and improves placement performance.

6. To Support Technical Skill Development

The platform also focuses on strengthening technical concepts and programming knowledge required during placement interviews. The Programming Language Questions Module helps students practice and revise commonly asked technical questions in different languages.

The objective is to improve students' technical competency in areas such as:

- C Programming
- C++
- Java
- Python
- JavaScript
- Data Structures and Algorithms

The platform enables students to strengthen conceptual understanding and improve coding interview readiness.

7. To Enable Self-Assessment and Performance Evaluation

The Personal Finance Tracker system includes quizzes and self-assessment modules that allow students to evaluate their preparation level.

The objective of this feature is to:

- Measure student understanding
- Identify strengths and weaknesses
- Encourage regular practice
- Improve confidence through assessment

Automated quiz evaluation helps students receive instant feedback and track their preparation progress effectively.

8. To Ensure Easy Accessibility and User-Friendly Interaction

Another important objective of the project is to provide a simple, responsive, and user-friendly interface so that students can use the system easily without technical difficulties.

The platform is designed with:

- Easy navigation
- Responsive layouts
- Clear module categorization
- Accessible dashboards
- Simple interaction workflows

This objective ensures that users can focus on learning and preparation rather than system complexity.

9. To Develop a Scalable and Future-Ready System

The Personal Finance Tracker platform is developed using a modular and scalable architecture so that future enhancements can be integrated easily.

The project aims to support future extensions such as:

- AI-based recommendation systems
- Resume analysis tools
- Performance analytics
- Personalized preparation roadmaps
- Institution-wide deployment
- Mobile application integration

The system is therefore designed not only for present requirements but also for long-term growth and scalability.

CHAPTER 4

Hardware and Software Requirements

4.1 HARDWARE REQUIREMENTS

Hardware and software requirements define the physical and technological resources necessary for the successful development, execution, and maintenance of a system. These requirements ensure that the project operates efficiently, delivers smooth performance, and supports all functionalities without technical limitations. For the AI-Powered Personal Finance Tracker, the hardware and software components have been selected carefully to support web application development, database management, AI-based processing, and data visualization. Since the project is developed using modern web technologies and machine learning techniques, it does not require extremely high-end infrastructure. Standard computing systems with moderate configurations are sufficient for development, testing, and deployment.

The project follows a client-server architecture where the frontend, backend, database, and AI modules interact with each other to provide an integrated financial management solution. Therefore, the hardware and software requirements are designed to ensure compatibility, scalability, and reliability throughout the system lifecycle.

4.2 Hardware Requirements

Hardware requirements refer to the physical devices and system configurations required to develop and run the AI-Powered Personal Finance Tracker effectively. The hardware must support multitasking operations such as frontend rendering, backend execution, database management, AI model processing, and testing activities. Since the application includes AI-based prediction models and OCR functionality, the system should provide sufficient processing power and memory for smooth execution.

4.2.1 Processor Requirements

The processor plays a crucial role in handling application logic, server-side processing, AI computations, and multitasking operations. A modern multi-core processor is recommended to ensure efficient performance during development and execution. The AI modules and OCR processing require additional computational capability for analyzing financial data and processing uploaded receipts.

For this project, a system equipped with an Intel Core i5 processor or AMD Ryzen 5 processor is considered suitable. Higher-end processors improve application responsiveness and reduce processing time for AI-based predictions and data analysis.

Multi-core processors are especially beneficial because they allow simultaneous execution of frontend tools, backend services, databases, and machine learning libraries without affecting performance.

Recommended configuration includes:

- Intel Core i5 / i7 or AMD Ryzen 5 / Ryzen 7
- Multi-core processing support
- Minimum clock speed of 2.5 GHz or higher

4.2.2 RAM Requirements

Random Access Memory (RAM) is essential for supporting multiple applications and services running simultaneously during development and execution. The AI-Powered Personal Finance Tracker requires sufficient memory to support:

- React frontend development
- Node.js backend server
- MongoDB database operations
- AI and machine learning computations
- OCR processing
- Browser-based testing

A minimum of 8 GB RAM is recommended for smooth execution of the system. However, 16 GB RAM is preferred for better multitasking and improved performance, especially when training AI models or handling large financial datasets. Adequate RAM ensures faster processing, reduced lag, and efficient debugging during development.

Recommended configuration includes:

- Minimum: 8 GB RAM
- Preferred: 16 GB RAM or higher

4.2.3 Storage Requirements

Storage is required for maintaining project files, source code, databases, AI models, uploaded receipts, and development tools. Since the system stores transaction records and OCR-processed data, efficient storage management is important for smooth performance.

Solid State Drives (SSDs) are preferred because they provide faster read/write speeds compared to traditional hard drives. Faster storage significantly improves application loading time, database access speed, and overall system responsiveness.

Recommended storage configuration includes:

- Minimum: 256 GB SSD
- Preferred: 512 GB SSD or higher

The additional storage space can also support future scalability and backup management.

4.2.4 Display and Resolution Requirements

A proper display setup is necessary for designing, testing, and monitoring the user interface of the application. Since the project includes dashboards, charts, and financial visualizations, higher screen resolution improves clarity and development efficiency.

A Full HD display is recommended because it allows developers to test responsive layouts and graphical components effectively. High-resolution displays also help in identifying UI inconsistencies and improving user experience.

Recommended display configuration includes:

- Minimum Resolution: 1366 × 768 pixels
- Preferred Resolution: 1920 × 1080 pixels (Full HD)

4.2.5 Input Devices

Standard input devices are necessary for coding, navigation, testing, and interaction with the development environment. A keyboard and mouse or touchpad are sufficient for project development and operation.

Required input devices include:

- Standard keyboard
- Optical mouse or touchpad

These devices support efficient coding, navigation, and system interaction.

4.2.6 Network Requirements

A stable internet connection is necessary for accessing online development resources, APIs, cloud services, version control systems, and deployment platforms. Internet connectivity also supports downloading libraries, testing APIs, integrating OCR services, and synchronizing databases.

The project may use cloud-based AI or OCR services, which require reliable network access. Additionally, collaboration through GitHub and remote deployment platforms depends on stable internet connectivity.

Recommended network configuration includes:

- Broadband or Wi-Fi connection
- Minimum speed: 10 Mbps
- Stable low-latency connection

4.3 Software Requirements

Software requirements refer to the operating systems, programming languages, frameworks, development tools, databases, and libraries required for designing, developing, and maintaining the AI-Powered Personal Finance Tracker. The selected software stack is based on scalability, compatibility, ease of development, and performance.

4.3.1 Operating System

The AI-Powered Personal Finance Tracker can be developed and executed on multiple operating systems because the selected technologies are cross-platform compatible. The operating system must support Node.js, MongoDB, frontend development tools, and AI libraries.

Supported operating systems include:

- Windows 10 / Windows 11
- Linux distributions such as Ubuntu
- macOS

These operating systems provide stable environments for both frontend and backend development.

4.3.2 Frontend Technologies

The frontend of the system is responsible for designing the visual interface and handling user interaction. Modern frontend technologies are used to create a responsive, interactive, and user-friendly experience.

React.js

React.js is used as the primary frontend library for building dynamic and reusable user interface components. It improves application performance through component-based architecture and virtual DOM rendering.

HTML

HTML is used to structure web pages and organize elements such as forms, navigation menus, dashboards, and financial reports.

CSS

CSS is used for styling the interface, improving responsiveness, typography, layouts, and overall visual appearance of the platform.

JavaScript

JavaScript provides dynamic functionality and interactive behavior, including form validation, chart rendering, API integration, and real-time updates.

Together, these technologies create a responsive and engaging user experience.

4.3.3 Backend Technologies

The backend is responsible for business logic, request processing, API handling, authentication, and communication with the database.

Node.js

Node.js is used as the runtime environment for executing server-side JavaScript code. It supports asynchronous operations and enables efficient handling of multiple requests simultaneously.

Express.js

Express.js is a lightweight Node.js framework used for:

- API development
- Routing
- Middleware integration
- Request handling

The backend architecture ensures efficient communication between frontend components and the database.

4.3.4 Database Management System

The database stores and manages all financial records, user profiles, OCR data, and AI-generated insights. MongoDB is used because of its flexibility and scalability.

The database stores:

- User information
- Income and expense records
- Uploaded receipt data
- OCR extracted information
- AI prediction results
- Financial reports

MongoDB supports efficient data retrieval and management while handling structured and semi-structured data effectively.

4.3.5 AI and Machine Learning Tools

The project integrates Artificial Intelligence and Machine Learning for expense prediction and financial analysis.

Technologies used include:

- Python libraries (optional integration)
- Scikit-learn
- TensorFlow / Keras
- Pandas and NumPy

These tools help analyze historical financial data and generate predictive insights.

4.3.6 OCR Technologies

OCR technology is used for extracting information from uploaded receipts and bills. Tesseract OCR or cloud-based OCR APIs may be used to process images and convert them into editable financial data.

OCR functionality reduces manual work and improves system automation.

4.3.7 Development Tools

Several development tools are used during coding, debugging, testing, and collaboration.

Visual Studio Code

Visual Studio Code is used as the primary code editor because of its lightweight environment and extensive extension support.

GitHub

GitHub is used for version control, code management, and collaboration among developers.

Postman

Postman is used for API testing and backend request validation.

MongoDB Compass

MongoDB Compass is used for database visualization and query execution.

These tools improve development efficiency and simplify project management.

4.3.8 Web Browsers

Modern web browsers are required for testing and accessing the platform.

Supported browsers include:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

These browsers ensure compatibility, responsiveness, and smooth interaction with the application.

CHAPTER 5

Project Flow

Project flow describes the overall working structure of the system and explains how different modules interact with each other to perform specific operations. It provides a clear understanding of how data moves through the application, how users interact with the system, and how the backend processes request to generate meaningful outputs. The project flow also demonstrates the sequence of operations followed within the system and highlights the integration between frontend components, backend services, databases, and AI modules.

For the AI-Powered Personal Finance Tracker, the project flow is designed to ensure smooth navigation, intelligent financial analysis, efficient data handling, and user-friendly interaction. The system follows a modular architecture where each module performs a specific function while remaining connected to the overall workflow of the application. The project integrates expense tracking, OCR-based bill processing, AI-powered predictions, financial visualization, and reporting functionalities into a unified platform.

The workflow begins with user authentication and continues through transaction management, data processing, AI analysis, and report generation. Every module communicates with the backend server and database to retrieve or store data efficiently. This structured workflow ensures that users can manage their finances effectively and receive meaningful financial insights.

Overview of Project Flow

The AI-Powered Personal Finance Tracker operates on a client-server architecture in which the frontend, backend, database, and AI components work together to provide a complete financial management solution. The frontend is responsible for user interaction and displaying information, while the backend handles request processing, business logic, and communication with the database. The database stores financial records, transaction details, user profiles, and AI-generated insights.

The overall project flow can be summarized as follows:

1. User accesses the platform
2. User authentication and verification
3. Dashboard loading and navigation
4. Income and expense management
5. Bill upload and OCR processing
6. Data storage in the database
7. AI-based expense analysis and prediction
8. Data visualization and reporting
9. User interaction and feedback

This workflow ensures smooth communication between different modules and allows users to perform financial management tasks efficiently.

5.3 User Authentication Flow

The project flow begins when the user accesses the AI-Powered Personal Finance Tracker through a web browser. The homepage or landing page acts as the entry point of the system. To ensure security and personalized access, the user must register or log in using valid credentials.

During the registration process, users provide basic information such as:

- Name
- Email address
- Password

The backend validates the entered information and stores user details securely in the database. Passwords are encrypted before storage to ensure data protection.

When a user logs in, the backend verifies the credentials by comparing them with the stored database records. If authentication is successful, the user is redirected to the dashboard. If invalid credentials are entered, the system displays an appropriate error message.

The authentication flow ensures:

- Secure access to user data
- Personalized dashboards
- Protection against unauthorized usage

This initial stage forms the foundation of the system's security and user management process.

5.4 Dashboard and Navigation Flow

The dashboard acts as the central interface of the AI-Powered Personal Finance Tracker. After successful login, users are redirected to the dashboard where they can access all functionalities of the system. The dashboard is designed to provide a clean and organized view of financial information.

The navigation flow allows users to move between different sections such as:

- Income management
- Expense tracking
- Bill uploads
- Financial reports

- AI predictions
- Data visualizations

When the user selects a specific module, the frontend sends a request to the backend server. The backend processes the request, retrieves relevant data from the database, and sends the response back to the frontend for display.

This navigation flow ensures:

- Smooth user experience
- Efficient module access
- Quick data retrieval

The dashboard also displays summaries such as:

- Total income
- Total expenses
- Remaining balance
- Monthly spending trends

This provides users with a quick overview of their financial status.

5.5 Income and Expense Management Flow

The income and expense management module is one of the core components of the system. This module enables users to record and manage their financial transactions efficiently.

Users can manually add:

- Income records
- Expense details
- Transaction categories
- Dates and descriptions

When the user submits transaction data, the frontend sends the information to the backend through APIs. The backend validates the entered data and stores it in the MongoDB database.

The system categorizes expenses into different sections such as:

- Food
- Transportation
- Entertainment
- Bills
- Shopping
- Healthcare

This categorization helps users analyze their spending patterns effectively.

Users can also:

- Edit transactions
- Delete records
- Filter transactions
- Search financial history

The transaction management flow ensures accurate data handling and efficient financial organization.

5.6 OCR-Based Bill Upload Flow

One of the advanced features of the AI-Powered Personal Finance Tracker is the OCR-based bill upload system. This module reduces manual effort by automatically extracting financial information from receipts and bills.

The flow begins when the user uploads an image of a bill or receipt through the application interface. The uploaded image is sent to the backend server for processing. The OCR engine then scans the image and extracts important information such as:

- Bill amount
- Date of purchase
- Merchant name
- Purchased items

After extraction, the backend processes and validates the data before storing it in the database. The extracted information is automatically categorized and added to the user's expense records.

This module improves:

- Automation
- Accuracy
- User convenience

The OCR-based flow significantly reduces manual data entry and simplifies expense tracking.

5.7 AI-Based Prediction and Analysis Flow

The AI-based prediction module is one of the most intelligent components of the system. This module analyzes historical financial data to identify spending patterns and predict future expenses.

The prediction flow begins when the backend retrieves past transaction records from the database. The AI model processes this data using machine learning algorithms such as:

- Linear Regression
- Time Series Forecasting
- ARIMA or LSTM models

The AI system analyzes:

- Monthly spending behavior
- Recurring expenses
- Category-wise trends
- Saving patterns

Based on this analysis, the system generates:

- Future expense predictions
- Budget recommendations
- Spending alerts
- Savings suggestions

The generated insights are then displayed on the dashboard in graphical and textual formats.

This module helps users:

- Plan budgets effectively
- Avoid overspending
- Improve financial discipline

The AI prediction flow transforms raw financial data into meaningful and actionable insights.

5.8 Data Visualization and Reporting Flow

The visualization module converts financial data into graphical representations to improve understanding and decision-making. The system generates charts, graphs, and reports using processed transaction data.

The backend retrieves financial data from the database and sends it to the frontend for visualization. The frontend displays:

- Pie charts for expense distribution
- Bar graphs for monthly comparisons
- Line charts for trend analysis

Users can generate reports for:

- Daily expenses
- Monthly spending
- Yearly financial summaries
- Category-wise analysis

This module enhances:

- Financial awareness
- User engagement
- Data interpretation

Visual reports help users quickly identify spending patterns and financial trends.

5.9 Backend and Database Interaction Flow

The backend acts as the central processing unit of the AI-Powered Personal Finance Tracker. It manages communication between the frontend and the database while handling all business logic.

Every user action such as:

- Adding transactions
- Uploading receipts
- Requesting predictions
- Viewing reports

is processed by the backend server.

The backend performs the following functions:

- Request validation
- Data processing
- AI integration
- Database communication
- API management
- Response generation

MongoDB stores all structured and semi-structured financial data, including:

- User profiles
- Transaction records
- OCR results
- AI-generated predictions
- Financial reports

Efficient backend and database interaction ensures:

- Fast performance
- Data consistency
- Reliable operation

This interaction flow is essential for maintaining smooth system functionality.

5.10 Error Handling and Security Flow

The system includes error-handling mechanisms to ensure reliability and stability. Validation is performed at both frontend and backend levels to prevent invalid data submission.

Examples include:

- Empty field validation
- Incorrect data format checks
- Authentication error handling
- API failure handling

Security mechanisms include:

- Password encryption
- Secure authentication
- Protected APIs
- Session management

These measures ensure data privacy and system reliability.

5.11 Overall System Workflow

The complete workflow of the AI-Powered Personal Finance Tracker integrates all modules into a unified system. Users begin by logging into the platform, managing their transactions, uploading bills, and viewing AI-generated insights. The backend processes all requests efficiently while the database stores and manages financial data securely.

The integration of OCR, AI prediction models, data visualization, and reporting creates a comprehensive financial management platform that enhances user experience and simplifies personal finance management.

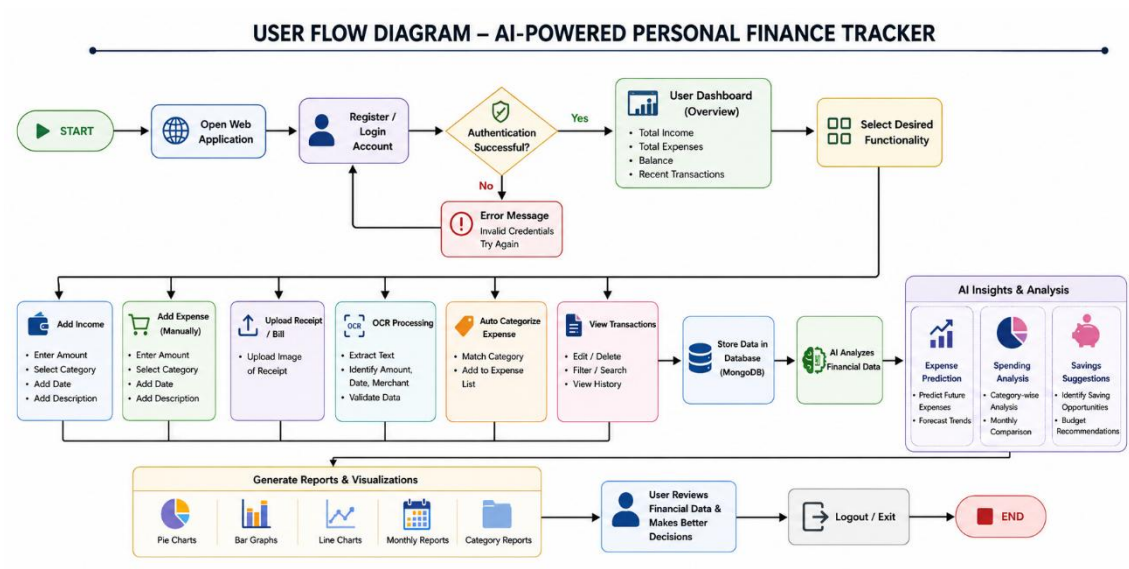


Figure 2: User Flow Diagram

CHAPTER 6

Project Outcome

The project outcome represents the final results achieved after the successful development and implementation of the AI-Powered Personal Finance Tracker. It highlights how the system fulfills its objectives, solves the identified problems, and provides value to users. The outcome of the project demonstrates the successful integration of modern web technologies, Artificial Intelligence, Machine Learning, and Optical Character Recognition to create an intelligent financial management platform.

The developed system enables users to efficiently manage their personal finances by tracking income and expenses, analyzing spending behavior, predicting future financial trends, and generating meaningful reports. The project successfully combines automation and intelligent analysis to improve user convenience and financial awareness.

The final outcome of the project reflects not only the technical implementation of the system but also its practical usefulness in real-world financial management scenarios. The project has been designed to provide a user-friendly and scalable solution that can be enhanced further in the future.

6.2 Successful Development of the System

The AI-Powered Personal Finance Tracker was successfully developed using the MERN stack architecture, which includes MongoDB, Express.js, React.js, and Node.js. The frontend interface was designed to provide smooth navigation and an intuitive user experience, while the backend efficiently handled request processing, business logic, and database communication.

The project successfully integrated multiple modules such as:

- User authentication
- Income and expense tracking
- OCR-based bill processing
- AI-powered expense prediction
- Data visualization and reporting

Each module was tested individually and integrated into a unified system to ensure smooth functionality and efficient performance.

The final system provides a centralized platform where users can manage their financial records efficiently and access meaningful insights regarding their financial activities.

6.3 Achievement of Project Objectives

The AI-Powered Personal Finance Tracker successfully achieved the objectives defined during the planning and design phases of the project. The system effectively automates financial tracking and reduces the dependency on manual methods. Users can add transactions, upload bills, and analyze their financial data through an interactive dashboard.

The implementation of OCR technology achieved the objective of automating expense entry by extracting transaction details directly from uploaded bills and receipts. This significantly reduced manual effort and improved data accuracy.

The AI-based prediction module successfully analyzed historical financial data and generated future expense predictions. This functionality enabled users to plan budgets more effectively and make informed financial decisions.

The visualization module also fulfilled its objective by presenting financial data through charts, graphs, and reports. These visual representations improved data understanding and helped users identify spending patterns and financial trends easily.

Overall, the system successfully met both functional and non-functional objectives by providing automation, security, efficiency, scalability, and user convenience.

6.4 Improved Financial Management

One of the major outcomes of the project is the improvement in personal financial management. The system allows users to maintain organized records of their income and expenses, making it easier to monitor financial activities regularly.

Users can:

- Track daily expenses
- Categorize transactions
- Monitor monthly spending
- Analyze financial trends
- Generate financial summaries

This structured management process improves financial awareness and encourages users to maintain better control over their spending habits.

The system also helps users identify unnecessary expenses and develop more disciplined budgeting practices. By providing detailed insights and predictive analysis, the platform supports smarter financial planning and long-term savings management.

6.5 Automation of Expense Tracking

The project successfully introduced automation into the financial management process through OCR-based bill processing. Instead of manually entering every expense, users can upload receipts or bills, and the system automatically extracts relevant information such as:

- Transaction amount
- Date
- Merchant details
- Purchased items

This automation reduces:

- Manual workload
- Human errors
- Time consumption

The OCR functionality improved the efficiency of the system and enhanced user convenience significantly. The integration of automation represents one of the most innovative outcomes of the project.

6.6 AI-Based Financial Insights

The implementation of Artificial Intelligence and Machine Learning models is another significant achievement of the project. The AI module successfully analyzed historical transaction data and generated intelligent financial insights.

The system provides:

- Expense predictions
- Spending pattern analysis
- Savings recommendations
- Budget planning suggestions

These AI-generated insights help users understand their financial behavior and make better decisions regarding future spending. The predictive capability of the system transforms it from a basic expense tracker into an intelligent financial assistant.

The AI module also contributes to improving financial discipline by alerting users about excessive spending and encouraging better savings practices.

6.7 Enhanced User Experience

The project outcome also includes the successful creation of a user-friendly and visually appealing interface. The frontend design ensures smooth navigation and easy accessibility for users with varying levels of technical knowledge.

The dashboard presents financial information in a clear and organized manner, allowing users to:

- Access reports quickly
- View charts and graphs
- Analyze transaction history
- Manage financial records easily

The responsive design ensures compatibility across different devices and screen sizes, improving accessibility and usability.

The system prioritizes simplicity and convenience, making financial management less complex and more engaging for users.

6.8 Secure Data Management

Security and privacy were important considerations throughout the development process. The system successfully implemented secure authentication mechanisms and protected user data from unauthorized access.

Key security measures include:

- Password encryption
- Authentication validation
- Protected APIs
- Secure database communication

These features ensure that sensitive financial information remains safe and confidential.

The successful implementation of security protocols increased system reliability and user trust.

6.9 Data Visualization and Reporting

The project outcome includes effective financial reporting and visualization features that help users understand their financial data more clearly. The system generates:

- Pie charts
- Bar graphs
- Line charts
- Monthly summaries
- Category-wise reports

These visual representations simplify complex financial information and improve analytical understanding.

Users can easily identify:

- Spending distribution
- High-expense categories
- Monthly financial trends
- Savings opportunities

The reporting module significantly enhances the usefulness and effectiveness of the system.

6.10 Practical Applications of the System

The AI-Powered Personal Finance Tracker has several practical applications in real-world scenarios. The system can be used by:

- Students
- Working professionals
- Freelancers
- Small business owners
- Families

It helps users manage daily financial activities efficiently while improving financial awareness and planning capabilities.

The project can also serve as a foundation for future financial management platforms and educational tools related to financial literacy.

6.11 Limitations of the Current System

Although the project successfully achieved its objectives, certain limitations still exist in the current version of the system.

The current system:

- Does not support direct bank account integration
- Relies on user-provided data for predictions
- Has limited offline functionality
- Requires internet connectivity for certain services

Additionally, OCR accuracy may vary depending on image quality and receipt formatting.

These limitations provide opportunities for future improvements and enhancements.

6.12 Future Enhancements

The project has strong potential for future expansion and feature enhancement. Several advanced functionalities can be integrated to improve the overall capability of the platform.

Future enhancements may include:

- Bank API integration
- Real-time transaction synchronization
- AI chatbot for financial advice
- Investment tracking module
- Tax calculation system
- Multi-user account management
- Mobile application development
- Multi-language support

The integration of advanced AI models and cloud-based services can further improve prediction accuracy and system scalability.

These future enhancements can transform the platform into a comprehensive digital financial ecosystem.

6.13 Overall Impact of the Project

The AI-Powered Personal Finance Tracker successfully demonstrates how modern technologies can be used to solve real-world financial management challenges. The project combines automation, artificial intelligence, machine learning, and data visualization to create a smart and efficient platform.

The system improves:

- Financial awareness
- Budget management
- Spending discipline
- Decision-making capabilities

It also reduces manual effort and simplifies financial tracking for users.

The project contributes positively to digital financial management practices and highlights the importance of AI-driven solutions in modern applications.

- Real-time mentorship and chat features

These improvements can further enhance the effectiveness and usability of the platform.

6.10. Implementation of Code:

```
import Image from "next/image";

import Link from "next/link";

import { Button } from "@components/ui/button";

import { Card, CardContent } from "@components/ui/card";

import {

  featuresData,

  howItWorksData,

  statsData,

  testimonialsData,

} from "@data/landing";

import { formatCurrency } from "@lib/utls";

const LandingPage = () => {
```

```

return (

  <div className="min-h-screen bg-stone-50 text-stone-900">

    <section className="border-b border-stone-200 bg-stone-50 pt-28">

      <div className="container mx-auto px-4 py-12 md:px-6 md:py-20">

        <div className="grid gap-12 lg:grid-cols-[1.1fr_0.9fr] lg:items-center">

          <div className="space-y-8">

            <div className="inline-flex items-center rounded-full border border-emerald-200 bg-emerald-50 px-4 py-2 text-xs font-semibold uppercase text-emerald-700">

              Built for personal finance clarity

            </div>

            <div className="space-y-5">

              <h1 className="max-w-4xl text-balance text-5xl font-semibold leading-tight md:text-7xl">

                Track income, expenses, and recurring payments without the clutter.

              </h1>

              <p className="max-w-2xl text-pretty text-lg leading-8 text-stone-600 md:text-xl">

                AI Finance Tracker gives you one calm workspace for accounts, transactions, reports, and day-to-day money decisions.

              </p>

            </div>

            <div className="flex flex-col gap-3 sm:flex-row">

              <Link href="/dashboard">

                <Button size="lg" className="h-12 px-6 bg-emerald-600 hover:bg-emerald-700">

                  Open Dashboard

                </Button>

              </Link>

            </div>

          </div>

        </div>

      </div>

    </section>

  </div>
)

```

```

    </Button>

  </Link>

  <Link href="#features">

    <Button size="lg" variant="outline" className="h-12 px-6">

      Explore Features

    </Button>

  </Link>

</div>

<div className="grid gap-4 sm:grid-cols-3">

  <Card className="border-stone-200 bg-white shadow-sm">

    <CardContent className="pt-6">

      <p className="text-sm text-stone-500">Daily workflow</p>

      <p className="mt-2 text-2xl font-semibold tabular-nums">CSV
import</p>

    </CardContent>

  </Card>

  <Card className="border-stone-200 bg-white shadow-sm">

    <CardContent className="pt-6">

      <p className="text-sm text-stone-500">Always visible</p>

      <p className="mt-2 text-2xl font-semibold tabular-nums">Live
balances</p>

    </CardContent>

  </Card>

  <Card className="border-stone-200 bg-white shadow-sm">

    <CardContent className="pt-6">

      <p className="text-sm text-stone-500">Made simpler</p>

```

```
        <p className="mt-2 text-2xl font-semibold tabular-nums">Recurring tracking</p>
```

```
    </CardContent>
```

```
  </Card>
```

```
</div>
```

```
</div>
```

```
<div className="rounded-[2rem] border border-stone-200 bg-white p-4 shadow-sm">
```

```
  <div className="rounded-[1.5rem] border border-stone-200 bg-stone-100 p-3">
```

```
    <div className="overflow-hidden rounded-[1.25rem] border border-stone-200 bg-white">
```

```
      <Image
```

```
        src="/images.jpg"
```

```
        alt="AI Finance Tracker interface preview"
```

```
        width={1200}
```

```
        height={800}
```

```
        className="h-auto w-full object-cover"
```

```
        priority
```

```
      />
```

```
    </div>
```

```
  </div>
```

```
<div className="mt-4 grid gap-3 sm:grid-cols-3">
```

```
  <div className="rounded-2xl bg-stone-100 p-4">
```

```
    <p className="text-xs uppercase text-stone-500">Monthly income</p>
```

```
    <p      className="mt-2      text-xl      font-semibold      tabular-nums">{formatCurrency(3870)}</p>
```

```
  </div>
```



```

<div className="rounded-2xl bg-stone-100 p-4">

  <p className="text-xs uppercase text-stone-500">Monthly spend</p>

  <p      className="mt-2      text-xl      font-semibold      tabular-
nums">{formatCurrency(1528)}</p>

</div>

<div className="rounded-2xl bg-emerald-50 p-4">

  <p className="text-xs uppercase text-emerald-700">Net flow</p>

  <p className="mt-2 text-xl font-semibold text-emerald-700 tabular-nums">
    +{formatCurrency(2342)}

  </p>

</div>

</div>

</div>

</div>

</section>

```

```

    </div>

  </div>

</section>

<section id="features" className="border-b border-stone-200 bg-stone-50">

  <div className="container mx-auto px-4 py-16 md:px-6 md:py-24">

    <div className="max-w-3xl space-y-4">

      <p className="text-sm font-semibold uppercase text-emerald-700">Core
features</p>

      <h2 className="text-balance text-4xl font-semibold md:text-5xl">

        A tighter workflow for the money decisions you make every week.

      </h2>

      <p className="text-pretty text-lg leading-8 text-stone-600">

        The product is designed around transaction tracking first, with clear reports and
        simple account management layered on top.

      </p>

    </div>

    <div className="mt-12 grid gap-4 md:grid-cols-2 xl:grid-cols-3">

      {featuresData.map((feature) => (

        <Card key={feature.title} className="border-stone-200 bg-white shadow-
sm">

          <CardContent className="space-y-4 pt-6">

            <div className="flex size-12 items-center justify-center rounded-2xl bg-
emerald-50 text-emerald-700">

              {feature.icon}

            </div>

            <div className="space-y-2">

```

```

    <h3 className="text-xl font-semibold">{feature.title}</h3>

    <p className="text-pretty leading-7 text-stone-600">

      {feature.description}

    </p>

  </div>

</CardContent>

</Card>

)}}

</div>

</div>

</section>

```

```

<section className="border-b border-stone-200 bg-white">

  <div className="container mx-auto px-4 py-16 md:px-6 md:py-24">

    <div className="grid gap-10 lg:grid-cols-[0.85fr_1.15fr]">

      <div className="space-y-4">

        <p className="text-sm font-semibold uppercase text-emerald-700">How it
works</p>

        <h2 className="text-balance text-4xl font-semibold md:text-5xl">

          Start simple, then layer in smarter tracking.

        </h2>

        <p className="text-pretty text-lg leading-8 text-stone-600">

          The onboarding path keeps things focused: create an account, add
transactions,

          and let the dashboard and analytics fill in around your real data.

        </p>

      </div>

    </div>

  </div>

```

```

<div className="grid gap-4 md:grid-cols-3">
  {howItWorksData.map((step, index) => (
    <Card key={step.title} className="border-stone-200 bg-stone-50 shadow-
sm">
      <CardContent className="space-y-4 pt-6">
        <div className="text-sm font-semibold uppercase text-emerald-700">
          Step {index + 1}
        </div>
        <h3 className="text-xl font-semibold">
          {step.title.split(". ")[1] || step.title}
        </h3>
        <p className="text-pretty leading-7 text-stone-600">
          {step.description}
        </p>
      </CardContent>
    </Card>
  )))
</div>
</div>
</div>
</section>

```

```

<section id="testimonials" className="border-b border-stone-200 bg-stone-50">
  <div className="container mx-auto px-4 py-16 md:px-6 md:py-24">
    <div className="max-w-3xl space-y-4">
      <p className="text-sm font-semibold uppercase text-emerald-700">What users
say</p>
      <h2 className="text-balance text-4xl font-semibold md:text-5xl">

```

People use it because it feels direct, not overwhelming.

</h2>

</div>

<div className="mt-12 grid gap-4 md:grid-cols-2 xl:grid-cols-3">

{testimonialsData.map((testimonial) => (

<Card key={testimonial.name} className="border-stone-200 bg-white shadow-sm">

<CardContent className="space-y-6 pt-6">

<div className="flex items-center gap-4">

<Image

src={testimonial.image}

alt={testimonial.name}

width={56}

height={56}

className="size-14 rounded-full object-cover"

/>

<div>

<p className="font-semibold">{testimonial.name}</p>

<p className="text-sm text-stone-500">{testimonial.role}</p>

</div>

</div>

<p className="text-pretty leading-7 text-stone-600">

“{testimonial.quote}”

</p>

</CardContent>

</Card>

```

    )))
  </div>

</div>

</section>

<section className="bg-white">
  <div className="container mx-auto px-4 py-16 md:px-6 md:py-24">
    <div className="rounded-[2rem] border border-stone-200 bg-stone-50 p-8 md:p-12">
      <div className="grid gap-8 lg:grid-cols-[1fr_auto] lg:items-end">
        <div className="space-y-4">
          <p className="text-sm font-semibold uppercase text-emerald-700">Ready to start</p>
          <h2 className="max-w-3xl text-balance text-4xl font-semibold md:text-5xl">
            Bring your accounts and transactions into one place.
          </h2>
          <p className="max-w-2xl text-pretty text-lg leading-8 text-stone-600">
            Import a CSV, add recurring entries, and start building a cleaner money routine.
          </p>
        </div>
        <div className="flex flex-col gap-3 sm:flex-row">
          <Link href="/dashboard">
            <Button size="lg" className="h-12 px-6 bg-emerald-600 hover:bg-emerald-700">
              Go to Dashboard
            </Button>
          </Link>

```

```

    <Link href="/transactions">

      <Button size="lg" variant="outline" className="h-12 px-6">

        Open Transactions

      </Button>

    </Link>

  </div>

</div>

</div>

</div>

</div>

</section>

</div>

);

};

export default LandingPage;
import { Suspense } from "react";

import { getAccountWithTransactions } from "@actions/account";

import { BarLoader } from "react-spinners";

import { TransactionTable } from "../_components/transaction-table";

import { notFound } from "next/navigation";

import { AccountChart } from "../_components/account-chart";

import { DatabaseUnavailable } from "@components/database-unavailable";

import { formatCurrency } from "@lib/utills";

export default async function AccountPage({ params }) {

  const resolvedParams = await params;

  const accountData = await getAccountWithTransactions(resolvedParams.id);

```

```

if (accountData?.isDatabaseUnavailable) {
  return (
    <DatabaseUnavailable
      title="Account data unavailable"
      description="This account page needs a live database connection, and the app
could not reach it right now."
    />
  );
}

if (!accountData) {
  notFound();
}

const { transactions, ...account } = accountData;

return (
  <div className="space-y-8 px-5">
    <div className="flex gap-4 items-end justify-between">
      <div>
        <h1 className="text-5xl sm:text-6xl font-bold tracking-tight gradient-title
capitalize">
          {account.name}
        </h1>
        <p className="text-muted-foreground">
          {account.type.charAt(0) + account.type.slice(1).toLowerCase()}{" "}
          Account

```



```

    </p>
  </div>

  <div className="text-right pb-2">
    <div className="text-xl sm:text-2xl font-bold">
      {formatCurrency(account.balance)}
    </div>
    <p className="text-sm text-muted-foreground">
      {account._count.transactions} Transactions
    </p>
  </div>
</div>

{/* Chart Section */}
<Suspense
  fallback={<BarLoader className="mt-4" width={"100%"} color="#9333ea" />}
>
  <AccountChart transactions={transactions} />
</Suspense>

{/* Transactions Table */}
<Suspense
  fallback={<BarLoader className="mt-4" width={"100%"} color="#9333ea" />}
>
  <TransactionTable transactions={transactions} />
</Suspense>
</div>

```

```

    );
  }

import { Badge } from "@components/ui/badge";

import {
  Card,
  CardContent,
  CardDescription,
  CardHeader,
  CardTitle,
} from "@components/ui/card";

import { AnalyticsFilters } from "../_components/analytics-filters";
import { getAnalyticsDetails } from "@actions/insights";
import { categoryColors } from "@data/categories";
import { formatCurrency } from "@lib/utils";

export const metadata = {
  title: "Analytics | AI Finance Tracker",
};

export default async function AnalyticsPage({ searchParams }) {
  const resolvedSearchParams = await searchParams;

  const analytics = await getAnalyticsDetails({
    months: resolvedSearchParams?.months,
    type: resolvedSearchParams?.type,
    accountId: resolvedSearchParams?.accountId,
    category: resolvedSearchParams?.category,
  });

```

```

return (
  <div className="space-y-8 px-5">
    <div className="flex items-center justify-between gap-4">
      <div>
        <h1 className="text-5xl sm:text-6xl font-bold tracking-tight gradient-title">
          Analytics
        </h1>
        <p className="text-muted-foreground mt-1">
          Find what is driving your spending, what changed from last month,
          and where you can improve next.
        </p>
      </div>
      <Badge variant="secondary">Actionable view</Badge>
    </div>

    <AnalyticsFilters
      transactionCount={analytics.transactionCount}
      activeCategories={analytics.activeCategories}
      accounts={analytics.accounts}
      categories={analytics.availableCategories}
      filters={analytics.filters}
    />

    <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
      <Card>
        <CardHeader className="pb-2">

```

```

    <CardDescription>Total Transactions</CardDescription>

    <CardTitle className="text-3xl">
        {analytics.transactionCount}
    </CardTitle>
</CardHeader>
</Card>

<Card>
    <CardHeader className="pb-2">
        <CardDescription>Active Categories</CardDescription>
        <CardTitle className="text-3xl">
            {analytics.activeCategories}
        </CardTitle>
    </CardHeader>
</Card>

<Card>
    <CardHeader className="pb-2">
        <CardDescription>Recurring Entries</CardDescription>
        <CardTitle className="text-3xl">
            {analytics.recurringTransactions.length}
        </CardTitle>
    </CardHeader>
</Card>

<Card>
    <CardHeader className="pb-2">
        <CardDescription>Largest Transaction</CardDescription>
        <CardTitle className="text-3xl">
            {formatCurrency(analytics.biggestTransactions[0]?.amount || 0)}

```

```

    </CardTitle>

  </CardHeader>

</Card>

</div>

<div className="grid gap-4 lg:grid-cols-3">
  {analytics.observations.length === 0 ? (
    <Card className="lg:col-span-3">
      <CardContent className="py-8 text-center text-sm text-muted-foreground">
        Add more transactions to unlock spending causes and improvement
        suggestions.
      </CardContent>
    </Card>
  ) : (
    analytics.observations.map((item) => (
      <Card
        key={item.title}
        className="border-cyan-200 bg-cyan-50/70 shadow-sm"
      >
        <CardHeader>
          <CardTitle className="flex items-center justify-between gap-3">
            <span>{item.title}</span>
            <Badge className="bg-cyan-600 text-white hover:bg-cyan-600">
              AI Insight
            </Badge>
          </CardTitle>
        </CardHeader>

```

```

<CardContent className="space-y-3 text-sm">
  <div>
    <p className="font-medium text-cyan-900">Cause</p>
    <p className="text-cyan-950/80">{item.cause}</p>
  </div>
  <div>
    <p className="font-medium text-cyan-900">Improve</p>
    <p className="text-cyan-950/80">{item.recommendation}</p>
  </div>
</CardContent>
</Card>
))
)}
</div>

```

```

<div className="grid gap-4 lg:grid-cols-2">
  <Card>
    <CardHeader>
      <CardTitle>Expense Drivers</CardTitle>
      <CardDescription>
        Categories contributing most to current spending.
      </CardDescription>
    </CardHeader>
    <CardContent>
      {analytics.categories.length === 0 ? (
        <div className="h-[240px] rounded-md border border-dashed bg-muted/30
flex items-center justify-center text-sm text-muted-foreground">

```

```

        No category data available
    </div>
  ): (
    <div className="space-y-3">
      {analytics.categories.map((item) => (
        <div
          key={item.name}
          className="flex items-center justify-between text-sm rounded-md border
p-3"
        >
          <div className="flex items-center gap-3 capitalize">
            <span
              className="h-2.5 w-2.5 rounded-full"
              style={{
                backgroundColor:
                  categoryColors[item.name] || "#64748b",
              }}
            />
            <p>{item.name}</p>
          </div>
          <p className="font-medium">{formatCurrency(item.amount)}</p>
        </div>
      )})
    </div>
  )}
</CardContent>
</Card>

```

```

<Card>

  <CardHeader>

    <CardTitle>Month Comparison</CardTitle>

    <CardDescription>

      Compare this month with the previous month.

    </CardDescription>

  </CardHeader>

  <CardContent>

    {analytics.monthComparison.currentExpense === 0 &&
    analytics.monthComparison.previousExpense === 0 ? (

      <div className="h-[240px] rounded-md border border-dashed bg-muted/30
flex items-center justify-center text-sm text-muted-foreground">

        No month-over-month data available

      </div>

    ) : (

      <div className="space-y-4 text-sm">

        <div className="rounded-md border p-4 flex items-center justify-between">

          <span className="text-muted-foreground">

            Current month expense

          </span>

          <span className="font-medium">

            {formatCurrency(analytics.monthComparison.currentExpense)}

          </span>

        </div>

        <div className="rounded-md border p-4 flex items-center justify-between">

          <span className="text-muted-foreground">

```



```

        Previous month expense
    </span>

    <span className="font-medium">
        {formatCurrency(analytics.monthComparison.previousExpense)}
    </span>
</div>

<div className="rounded-md border p-4 flex items-center justify-between">
    <span className="text-muted-foreground">Difference</span>
    <span
        className={`font-medium ${
            analytics.monthComparison.delta > 0
                ? "text-amber-600"
                : "text-emerald-600"
        }}
    >
        {analytics.monthComparison.delta > 0 ? "+" : "-"}
        {formatCurrency(Math.abs(analytics.monthComparison.delta))}
    </span>
</div>
</div>

)}
</CardContent>
</Card>
</div>

<Card>
    <CardHeader>

```

```

<CardTitle>Likely Subscription Burden</CardTitle>

<CardDescription>

  Repeated or persistent charges that may behave like subscriptions.

</CardDescription>

</CardHeader>

<CardContent>

  {analytics.likelySubscriptions.length === 0 ? (

    <div className="rounded-md border border-dashed bg-muted/30 p-6 text-sm
text-muted-foreground text-center">

      No subscription-like spending detected in the selected range

    </div>

  ) : (

    <div className="space-y-3">

      {analytics.likelySubscriptions.map((item) => (

        <div

          key={item.name}

          className="rounded-md border p-3 flex items-center justify-between text-
sm"

        >

          <div>

            <p className="font-medium">{item.name}</p>

            <p className="text-muted-foreground capitalize">

              {item.count} charges • {item.category}

            </p>

          </div>

          <div>

            <p className="font-medium">{formatCurrency(item.amount)}</p>

          </div>

        </div>

      )
    )
  )
}

```

```

        </div>

    )}

</CardContent>

</Card>

<Card>

  <CardHeader>

    <CardTitle>Account Activity</CardTitle>

    <CardDescription>

      Total transaction volume grouped by account.

    </CardDescription>

  </CardHeader>

  <CardContent>

    {analytics.accountBreakdown.length === 0 ? (

      <div className="h-[180px] rounded-md border border-dashed bg-muted/30
flex items-center justify-center text-sm text-muted-foreground">

        Add transactions to see account-level analytics

      </div>

    ) : (

      <div className="space-y-3">

        {analytics.accountBreakdown.map((value) => (

          <div

            key={value.name}

            className="rounded-md border p-3 text-sm flex items-center justify-
between"

          >

            <div>

              <p className="font-medium">{value.name}</p>

```

```

        <p className="text-muted-foreground">
            {value.count} transactions
        </p>
    </div>

    <p className="text-muted-foreground">
        {formatCurrency(value.total)}
    </p>
</div>

    )}
</div>

)}
</CardContent>
</Card>

<div className="grid gap-4 lg:grid-cols-2">
    <Card>
        <CardHeader>
            <CardTitle>Largest Transactions</CardTitle>
            <CardDescription>
                Biggest entries across your current dataset.
            </CardDescription>
        </CardHeader>
        <CardContent>
            {analytics.biggestTransactions.length === 0 ? (
                <div className="rounded-md border border-dashed bg-muted/30 p-6 text-sm
text-muted-foreground text-center">
                    No transaction data available yet

```

```

</div>

): (
<div className="space-y-3">
  {analytics.biggestTransactions.map((transaction) => (
    <div
      key={transaction.id}
      className="rounded-md border p-3 flex items-center justify-between text-
sm"
    >
      <div>
        <p className="font-medium">
          {transaction.description || transaction.category}
        </p>
        <p className="text-muted-foreground">
          {transaction.account?.name || "Account"} • {" "}
          {transaction.type}
        </p>
      </div>
      <p className="font-medium">
        {formatCurrency(transaction.amount)}
      </p>
    </div>
  )})
</div>

)}

</CardContent>

</Card>

```

```

<Card>

  <CardHeader>

    <CardTitle>Repeated Expense Hotspots</CardTitle>

    <CardDescription>

      Repeated spending patterns that may be worth reviewing.

    </CardDescription>

  </CardHeader>

  <CardContent>

    {analytics.topRepeatedExpenses.length === 0 ? (

      <div className="rounded-md border border-dashed bg-muted/30 p-6 text-sm
text-muted-foreground text-center">

        No repeated expense pattern found yet

      </div>

    ) : (

      <div className="space-y-3">

        {analytics.topRepeatedExpenses.map((item) => (

          <div

            key={item.name}

            className="rounded-md border p-3 flex items-center justify-between text-
sm"

          >

            <div>

              <p className="font-medium">{item.name}</p>

              <p className="text-muted-foreground">

                {item.count} transactions

              </p>

            </div>

          </div>

        )

      )

    )

  </CardContent>

</Card>

```

```

        <p className="font-medium">{formatCurrency(item.amount)}</p>
      </div>
    )))
  </div>
})
</CardContent>
</Card>
</div>
</div>
);
import { getDashboardOverview } from "@actions/dashboard";
import { AccountCard } from "../_components/account-card";
import { CreateAccountDrawer } from "@components/create-account-drawer";
import { BudgetProgress } from "../_components/budget-progress";
import { Badge } from "@components/ui/badge";
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@components/ui/card";
import {
  AlertTriangle,
  ArrowDownRight,
  ArrowUpRight,
  Landmark,
  Plus,
  Wallet,
} from "lucide-react";
import { DashboardOverview } from "../_components/transaction-overview";
import { formatCurrency } from "@lib/utls";

```

```

export default async function DashboardPage() {
  const dashboard = await getDashboardOverview();
  const { accounts, transactions, budgetData, summary, alerts } = dashboard;

  return (
    <div className="space-y-8">
      <div className="flex items-center justify-between gap-4 flex-wrap">
        <div>
          <h2 className="text-3xl sm:text-4xl font-bold tracking-tight">
            Welcome back
          </h2>
          <p className="text-muted-foreground mt-1">
            Track balances, spending, and recent movement from one place.
          </p>
        </div>
        <Badge variant="secondary">
          {accounts.length} account{accounts.length === 1 ? "" : "s"}
        </Badge>
      </div>

      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        <Card>
          <CardHeader className="pb-2">
            <CardDescription>Total Balance</CardDescription>
            <CardTitle className="text-3xl">
              {formatCurrency(summary.totalBalance)}
            </CardTitle>
          </CardHeader>
        </Card>
      </div>
    </div>
  );
}

```



```

        </CardTitle>

    </CardHeader>

    <CardContent className="flex items-center gap-2 text-sm text-muted-foreground">

        <Wallet className="h-4 w-4" />

        Across all linked accounts

    </CardContent>

</Card>

<Card>

    <CardHeader className="pb-2">

        <CardDescription>Total Income</CardDescription>

        <CardTitle className="text-3xl text-green-600">

            {formatCurrency(summary.totalIncome)}

        </CardTitle>

    </CardHeader>

    <CardContent className="flex items-center gap-2 text-sm text-muted-foreground">

        <ArrowUpRight className="h-4 w-4" />

        Captured transactions

    </CardContent>

</Card>

<Card>

    <CardHeader className="pb-2">

        <CardDescription>Total Expense</CardDescription>

        <CardTitle className="text-3xl text-red-500">

            {formatCurrency(summary.totalExpense)}

        </CardTitle>

    </CardHeader>

```

```
<CardContent className="flex items-center gap-2 text-sm text-muted-foreground">
```

```
<ArrowDownRight className="h-4 w-4" />
```

```
Captured transactions
```

```
</CardContent>
```

```
</Card>
```

```
<Card>
```

```
<CardHeader className="pb-2">
```

```
<CardDescription>Net Flow</CardDescription>
```

```
<CardTitle
```

```
className={`text-3xl ${
```

```
summary.monthlyNet >= 0 ? "text-emerald-600" : "text-amber-600"
```

```
}`}>
```

```
>
```

```
{summary.monthlyNet >= 0 ? "+" : "-"}
```

```
{formatCurrency(Math.abs(summary.monthlyNet))}
```

```
</CardTitle>
```

```
</CardHeader>
```

```
<CardContent className="flex items-center gap-2 text-sm text-muted-foreground">
```

```
<Landmark className="h-4 w-4" />
```

```
Income minus expense
```

```
</CardContent>
```

```
</Card>
```

```
</div>
```

```
{alerts.length > 0 && (
```

```
<div className="grid gap-4 lg:grid-cols-2">
```

```

    {alerts.map((alert) => (
      <Card key={alert.title} className="border-amber-200 bg-amber-50/50">
        <CardHeader className="pb-2">
          <CardTitle className="flex items-center gap-2 text-base">
            <AlertTriangle className="h-4 w-4 text-amber-600" />
            {alert.title}
          </CardTitle>
        </CardHeader>
        <CardContent className="text-sm text-muted-foreground">
          {alert.body}
        </CardContent>
      </Card>
    ))}
  </div>
)}

```

```

{/* Budget Progress */}

<BudgetProgress
  initialBudget={budgetData?.budget}
  currentExpenses={budgetData?.currentExpenses || 0}
/>

```

```

{/* Dashboard Overview */}

<DashboardOverview accounts={accounts} transactions={transactions} />

```

```

{/* Accounts Grid */}

<div className="space-y-4">

```

```

<div className="flex items-center justify-between gap-4">

  <div>

    <h3 className="text-2xl font-semibold tracking-tight">Accounts</h3>

    <p className="text-sm text-muted-foreground">

      Switch defaults, review balances, and jump into each account.

    </p>

  </div>

</div>

<div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">

  <CreateAccountDrawer>

    <Card className="hover:shadow-md transition-shadow cursor-pointer border-dashed">

      <CardContent className="flex flex-col items-center justify-center text-muted-foreground h-full pt-5">

        <Plus className="h-10 w-10 mb-2" />

        <p className="text-sm font-medium">Add New Account</p>

      </CardContent>

    </Card>

  </CreateAccountDrawer>

  {accounts.length > 0 &&

    accounts?.map((account) => (

      <AccountCard key={account.id} account={account} />

    ))}

</div>

</div>

</div>

);

```

```

}
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card";

import { Badge } from "@/components/ui/badge";

import { ReportsSummaryCards } from "../_components/summary-cards";

import { ReportsInsightPanel } from "../_components/insight-panel";

import { getReportsOverview } from "@/actions/insights";

import { getUserTransactions } from "@/actions/transaction";

import { formatCurrency } from "@/lib/utls";

export const metadata = {
  title: "Reports | AI Finance Tracker",
};

export default async function ReportsPage() {
  const report = await getReportsOverview();

  const transactionsResult = await getUserTransactions();

  const transactions = transactionsResult?.success ? transactionsResult.data : [];

  return (
    <div className="space-y-8 px-5">
      <div className="flex items-center justify-between gap-4">
        <div>
          <h1 className="text-5xl sm:text-6xl font-bold tracking-tight gradient-title">
            Reports
          </h1>
          <p className="text-muted-foreground mt-1">
            Monthly summaries and AI insights for your financial activity.

```

```

    </p>

  </div>

  <Badge variant="secondary">{report.monthLabel}</Badge>

</div>

<ReportsSummaryCards report={report} />

<div className="grid gap-4 lg:grid-cols-3">
  <Card className="lg:col-span-2">
    <CardHeader>
      <CardTitle>Income vs Expense Trend</CardTitle>
      <CardDescription>
        Last six months of cashflow totals.
      </CardDescription>
    </CardHeader>
    <CardContent>
      {report.trend.length === 0 ? (
        <div className="h-[280px] rounded-md border border-dashed bg-muted/30
flex items-center justify-center text-sm text-muted-foreground">
          No trend data available yet
        </div>
      ) : (
        <div className="space-y-3">
          {report.trend.map((item) => (
            <div
              key={item.label}
              className="rounded-md border p-3 flex items-center justify-between text-
sm"

```

```

    >
    <p className="font-medium">{item.label}</p>
    <div className="flex items-center gap-4 text-muted-foreground">
      <span>Income: {formatCurrency(item.income)}</span>
      <span>Expense: {formatCurrency(item.expense)}</span>
    </div>
  </div>
  )}
</div>
)}
</CardContent>
</Card>

<ReportsInsightPanel insights={report.insights} />
</div>

<Card>
  <CardHeader>
    <CardTitle>Recent Activity</CardTitle>
    <CardDescription>
      Your latest transactions across all accounts.
    </CardDescription>
  </CardHeader>
  <CardContent>
    {transactions.length === 0 ? (
      <div className="rounded-md border border-dashed bg-muted/30 p-6 text-sm
text-muted-foreground text-center">

```

No transactions yet. Add a few entries and this report will start showing trends and recent activity.

```
    </div>

  ): (
    <div className="space-y-3">
      {transactions.slice(0, 8).map((transaction) => (
        <div
          key={transaction.id}
          className="rounded-md border p-3 flex items-center justify-between text-
sm"
        >
          <div>
            <p className="font-medium">
              {transaction.description || transaction.category}
            </p>
            <p className="text-muted-foreground">
              {transaction.account?.name || "Account"} • {transaction.category}
            </p>
          </div>
          <div className="text-right">
            <p
              className={
                transaction.type === "EXPENSE"
                ? "font-medium text-red-500"
                : "font-medium text-green-600"
              }
            >
              {transaction.type === "EXPENSE" ? "-" : "+"}
```



```

        {formatCurrency(transaction.amount)}
      </p>
      <p className="text-muted-foreground">
        {new Date(transaction.date).toLocaleDateString("en-US")}
      </p>
    </div>
  </div>

  )}
</div>

)}
</CardContent>
</Card>
</div>

);
}

import { Badge } from "@components/ui/badge";
import { getUserTransactions } from "@actions/transaction";
import { TransactionsManager } from "../_components/transactions-manager";

export const metadata = {
  title: "Transactions | AI Finance Tracker",
};

export default async function TransactionsPage() {
  const transactionsResult = await getUserTransactions();
  const transactions = transactionsResult?.success ? transactionsResult.data : [];

```

```

return (
  <div className="space-y-8 px-5">
    <div className="flex items-center justify-between gap-4">
      <div>
        <h1 className="text-5xl sm:text-6xl font-bold tracking-tight gradient-title">
          Transactions
        </h1>
        <p className="text-muted-foreground mt-1">
          Manage imports, exports, recurring entries, and your full transaction history.
        </p>
      </div>
      <Badge variant="secondary">CSV Ready</Badge>
    </div>

    <TransactionsManager transactions={transactions} />
  </div>
);

```

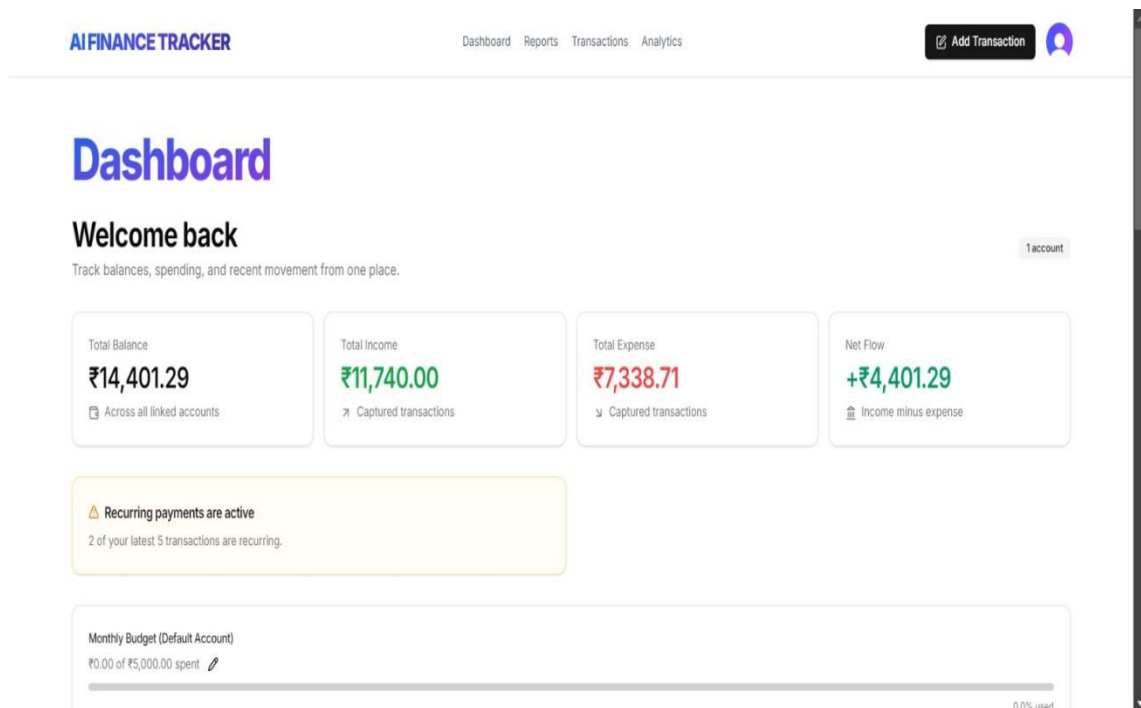


Figure 3: Dashboard Page

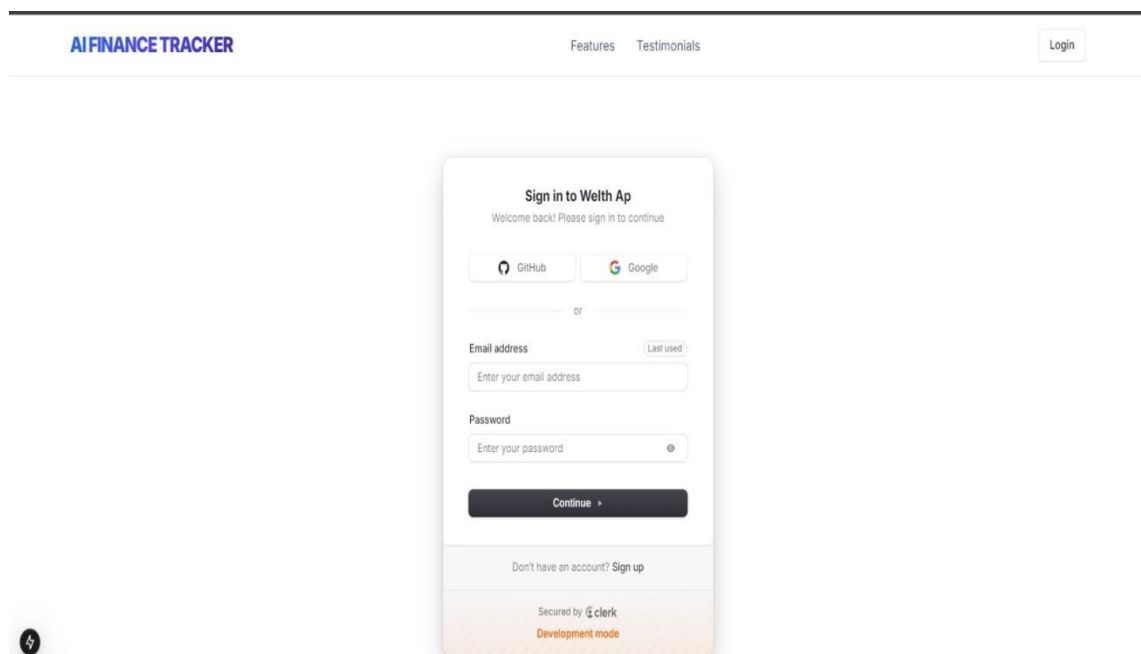


Figure 4: Login Page

AI FINANCE TRACKER

[Features](#)
[Testimonials](#)

Login

Create your account

Welcome! Please fill in the details to get started.

GitHub
 Google

or

First name

Optional

Last name

Optional

First name

Last name

Email address

Enter your email address

Password

Enter your password

Your password must contain 8 or more characters.

Continue

Already have an account? Sign in

Secured by 16 checks

Figure 5: Sign Up Page

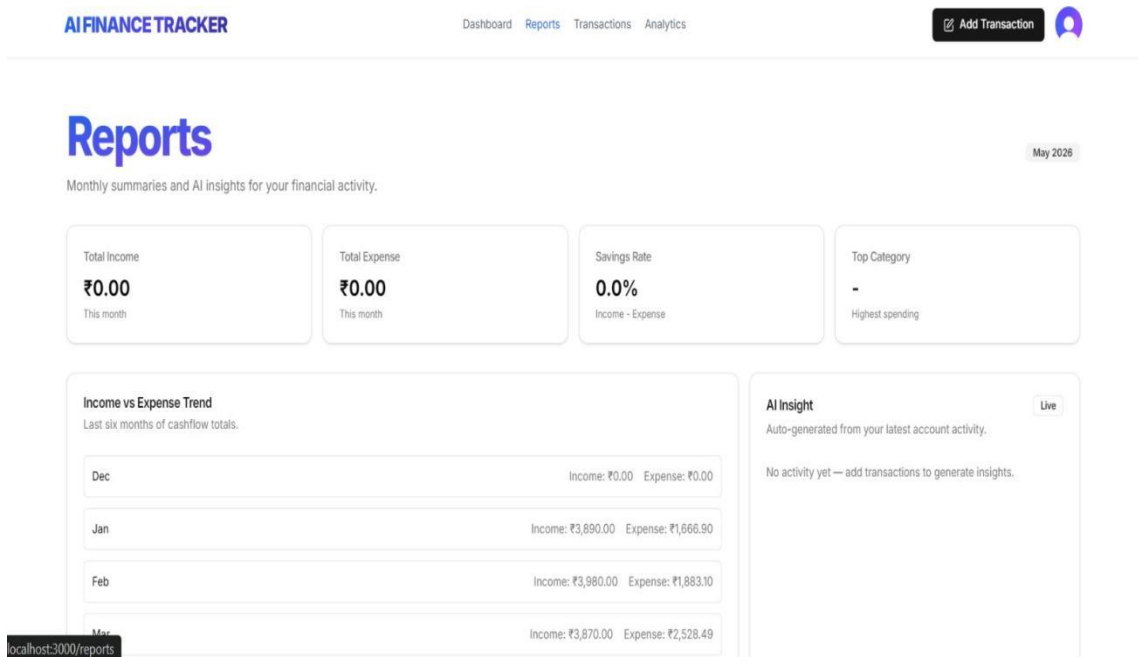


Figure 6: Reports Screen

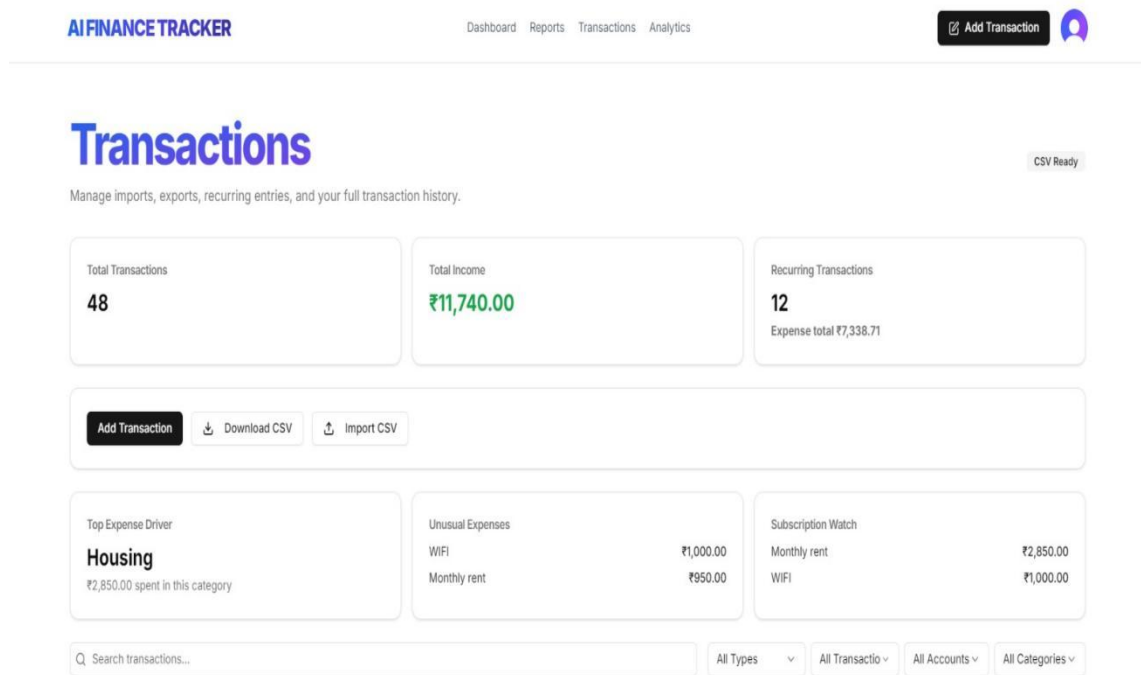


Figure 7: Transactions Page

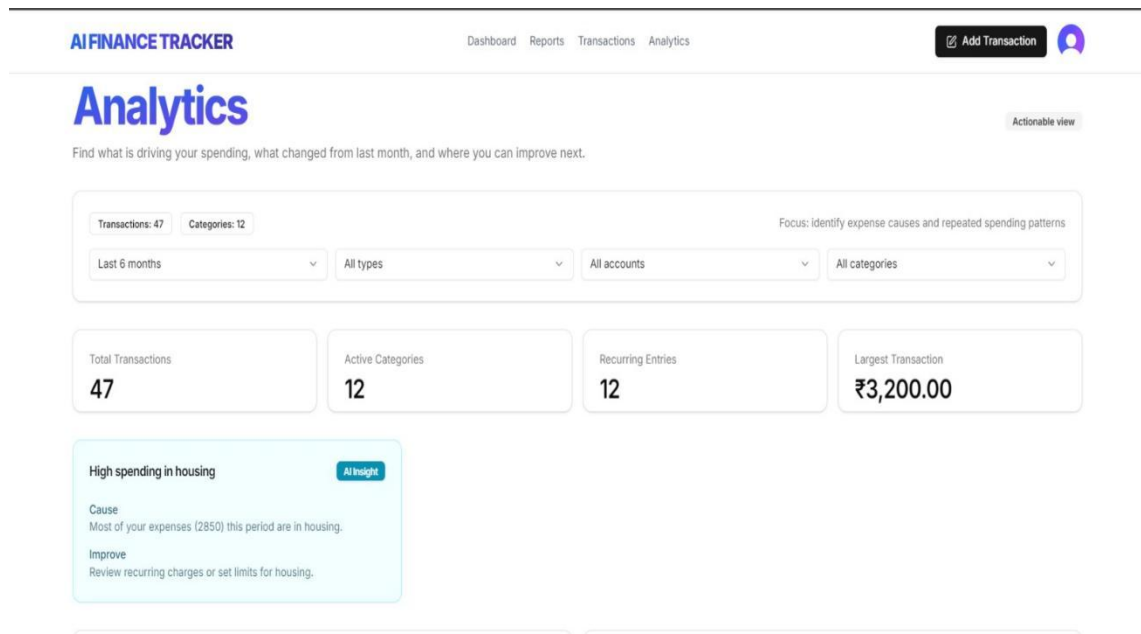


Figure 8 Analytics Page

AI FINANCE TRACKER

DashboardReportsTransactionsAnalytics

Add Transaction

Add Transaction

Scan Receipt with AI

Type

Expense

Amount

0.00

Account

main (₹14,401.29)

Category

Select category

Date

May 20th, 2026

Description

Enter description

Recurring Transaction

Figure 9 Add Transaction Page

CHAPTER 7

References and Bibliography

7.1 Books and Academic References

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson Education, 4th Edition, 2020.
3. Ethem Alpaydin, *Introduction to Machine Learning*, MIT Press, 4th Edition, 2020.
4. David Flanagan, *JavaScript: The Definitive Guide*, O'Reilly Media, 7th Edition, 2020.
5. Alex Banks and Eve Porcello, *Learning React: Modern Patterns for Developing React Apps*, O'Reilly Media, 2020.
6. Brad Dayley, Brendan Dayley, and Caleb Dayley, *Node.js, MongoDB and Angular Web Development*, Addison-Wesley Professional, 2018.
7. Robin Nixon, *Learning PHP, MySQL & JavaScript*, O'Reilly Media, 6th Edition, 2021.
8. Simon Holmes, *Getting MEAN with Mongo, Express, Angular, and Node*, Manning Publications, 2019.
9. Kurose and Ross, *Computer Networking: A Top-Down Approach*, Pearson Education, 8th Edition, 2021.
10. Thomas H. Davenport and Rajeev Ronanki, *Artificial Intelligence for the Real World*, Harvard Business Review Press, 2018.

7.2 Research Papers and Journals

1. Brown, T., et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
2. LeCun, Y., Bengio, Y., and Hinton, G., "Deep Learning," *Nature Journal*, Vol. 521, 2015.
3. Smith, J., and Anderson, P., "AI-Based Financial Forecasting Models for Personal Finance Applications," *International Journal of Computer Applications*, 2021.
4. Kumar, R., and Sharma, A., "Optical Character Recognition Techniques in Financial Systems," *International Journal of Advanced Research in Computer Science*, 2020.

5. Patel, N., "Machine Learning Approaches for Expense Prediction and Financial Analytics," *Journal of Data Science and Artificial Intelligence*, 2022.
6. Wang, H., et al., "Personal Finance Management using Artificial Intelligence," *IEEE International Conference on Intelligent Systems*, 2021.
7. Gupta, S., and Verma, P., "Data Visualization Techniques for Financial Analytics," *International Journal of Information Technology*, 2022.
8. Zhao, L., "Financial Behavior Prediction using Machine Learning Algorithms," *Journal of Artificial Intelligence Research*, 2021.

7.3 Online Learning Platforms and Tutorials

1. W3Schools – Web Development Tutorials
<https://www.w3schools.com/>
2. GeeksforGeeks – Programming and Development Resources
<https://www.geeksforgeeks.org/>
3. FreeCodeCamp – MERN Stack Development Tutorials
<https://www.freecodecamp.org/>
4. Coursera – Machine Learning and AI Courses
<https://www.coursera.org/>
5. Udemy – React and Node.js Development Courses
<https://www.udemy.com/>
6. YouTube Educational Tutorials for MERN Stack and AI Development
7. Stack Overflow – Technical Discussions and Error Resolution
<https://stackoverflow.com/>

7.4 Websites and Online Resources

1. <https://www.mongodb.com/>
2. <https://react.dev/>
3. <https://nodejs.org/>
4. <https://expressjs.com/>
5. <https://www.tensorflow.org/>
6. <https://scikit-learn.org/>
7. <https://github.com/>
8. <https://developer.mozilla.org/>
9. <https://www.geeksforgeeks.org/>
10. <https://stackoverflow.com/>
11. <https://www.w3schools.com/>
12. <https://tesseract-ocr.github.io/>

7.5 Bibliography

1. Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT Press, 2016.
2. Russell, S., and Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson Education, 2020.
3. Flanagan, D. *JavaScript: The Definitive Guide*. O'Reilly Media, 2020.
4. Holmes, S. *Getting MEAN with Mongo, Express, Angular, and Node*. Manning Publications, 2019.
5. Alpaydin, E. *Introduction to Machine Learning*. MIT Press, 2020.
6. Dayley, B., Dayley, B., and Dayley, C. *Node.js, MongoDB and Angular Web Development*. Addison-Wesley Professional, 2018.
7. Official documentation and online resources from React.js, MongoDB, Node.js, Express.js, TensorFlow, and Scikit-learn.